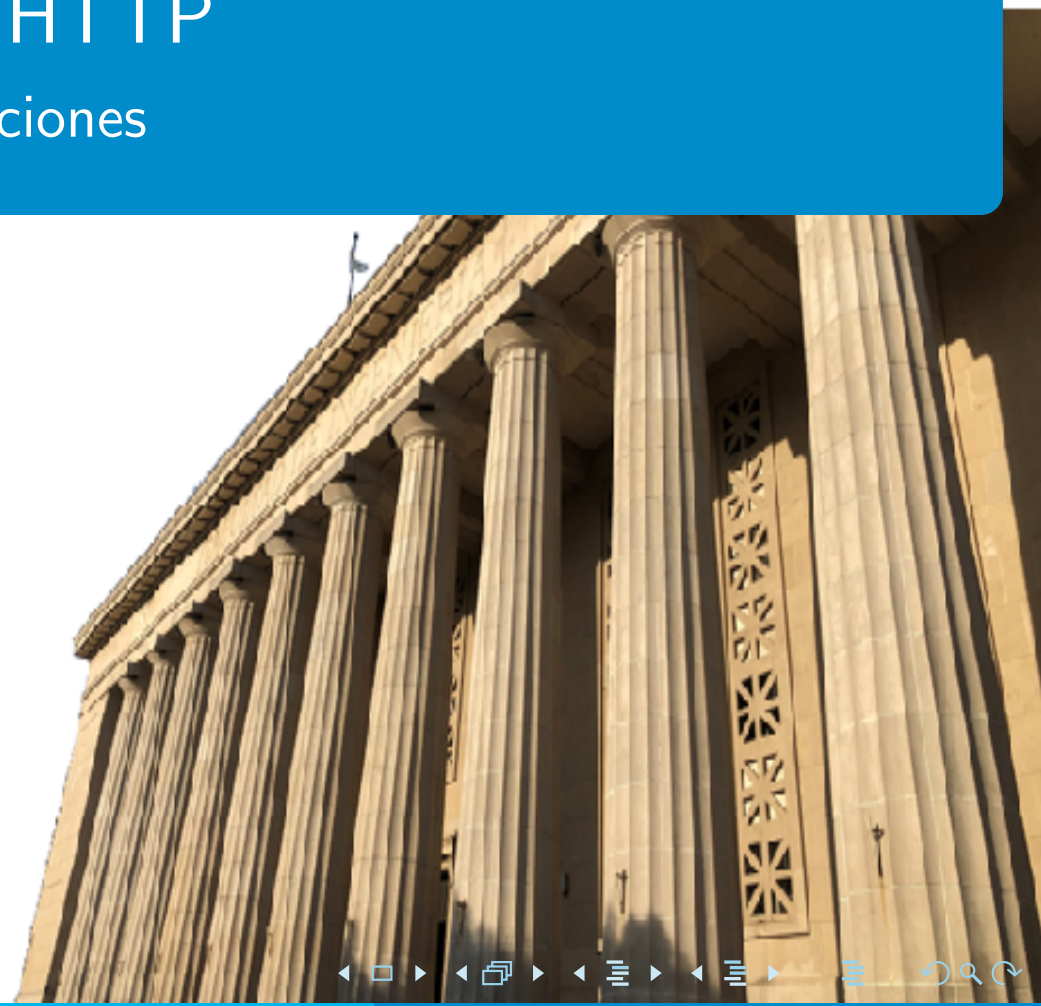
Nivel de Aplicación - Características generales - HTTP

86.12/TB067 Redes de Comunicaciones

Silvia Lovato

Facultad de Ingeniería
UBA

28 de agosto de 2024



Contenidos

1. Generalidades

2. HTTP

A continuación...

1. Generalidades

2. HTTP

Aplicaciones

- Aplicaciones que ofrecen un servicio.
- Involucra una comunicación entre las partes.
- Se soporta sobre distintos SO.[1]
- Se ejecuta en los extremos.
- Ejemplos: HTTP, Mail, chat, streaming, etc.

[1] Diferenciar comunicación entre procesos de la misma estación o entre estaciones.

Paradigma cliente-servidor

- Involucra una comunicación entre el cliente que solicita el servicio y el servidor que lo ofrece.
- Los roles pueden ser diferenciados:
  - ➔ Servidor: siempre disponible
  - ➔ Cliente: se conecta bajo demanda; no se conectan entre sí
- Ejemplos: HTTP, DNS, FTP, NTP, Telnet, SSH, etc.

Paradigma peer-to-peer

- Servidor ni siempre disponible.
- Se diluye o desaparecen los roles diferenciados.
- Los clientes solicitan atención a otros clientes.
- Se requiere una gestión más compleja.
- Ejemplos: Descarga de SW de servidores distribuidos.

Socket

Socket

- Análogo a una puerta, interface de contacto entre el proceso lado local y el protocolo que envía los mensajes.
- Involucra la dirección IP de la estación, el protocolo de transporte y número de port.
- Hay uno en cada extremo de la comunicación.
- Se identifica de forma similar a un número de proceso a nivel del SO.

Ejemplo: TCP; 157.92.49.38; 80 (lado servidor)

TCP; 122.36.99.208; 9887 (lado cliente)

Sesión

Sesión

- Establece la conexión de sockets entre extremos, sea punto-a.punto o cliente-servidor.
- Es el ámbito de la comunicación interactiva entre los extremos.
- Incluye el protocolo que otorga el reglamento y el lenguaje para la tarea a completar; y el tiempo que ocupa la transacción.
- La comunicación es bidireccional.
- Quíntupla de números del nivel de transporte y de red.

Ejemplo: (UDP; 208.67.222.222; 23; 161.190.0.33; 23055).

Requerimientos de las aplicaciones

Cada aplicación tiene sus requerimientos particulares, que reclamará al nivel de transporte. Estos son algunos, otros pueden ser la integridad, el jitter, etc. [2]

Aplicación	**pérdida de datos**	**Throughput**	**sensib t. de entrega**
Transf. de archivo Correo electr. tráfico web	sin pérdida	dinámico	no
Audio t.real Video t.real	tolerante	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	sí, $< 10ms$
streaming audio/video	tolerante	ídem anterior	sí, pocos seg.
juego online	tolerante	*Kbps*	sí, dec ms
texto, mensajería	sin pérdida	dinámico	sí/no

[2] Cuadro: Kurose, Ross, 2020.

Variantes de protocolos de transporte

- UDP: User Datagram Protocol
  - simple
  - no confiable
  - Aplic ej: DNS
- TCP: Transport Control Protocol
  - complejo
  - confiable
  - Aplic ej: HTTP
- TLS: Transport Layer Security
  - aporta seguridad a TCP
  - implementación a nivel de aplicación vs. API

Qué define el nivel de aplicación

Tipos de mensajes intercambiados

request, response

Sintaxis de los mensajes

- Cuáles son los campos de los mensajes

Semántica de los mensajes

- Qué significa el contenido de los campos

Reglas

- sobre cuándo y cómo los procesos envían y responden a los mensajes

Protocolos abiertos

- Definidos en RFCs a libre y abierta disponibilidad
- Permiten la interoperatividad

Ejemplos: SMTP, HTTP

Protocolos propietarios

Ejemplo: Skype

A continuación...

1. Generalidades

2. HTTP

Web

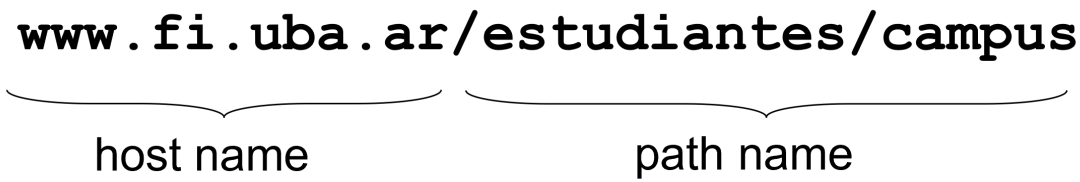
WWW: World Wide Web

Sistema de información que permite el intercambio de contenidos sobre Internet. Los contenidos pueden ser documentos HTML u otros como imágenes o audio entre otros.

- Los contenidos pueden ser de tipo archivo HTML, imagen JPEG, audio MP4, etc.
- Incluso admite MIME *(continuará...)*

URL: Universal Resource Locator

Tanto los servidores como los contenidos son identificados y ubicados mediante una URL, a modo de dirección. RFC 1738.

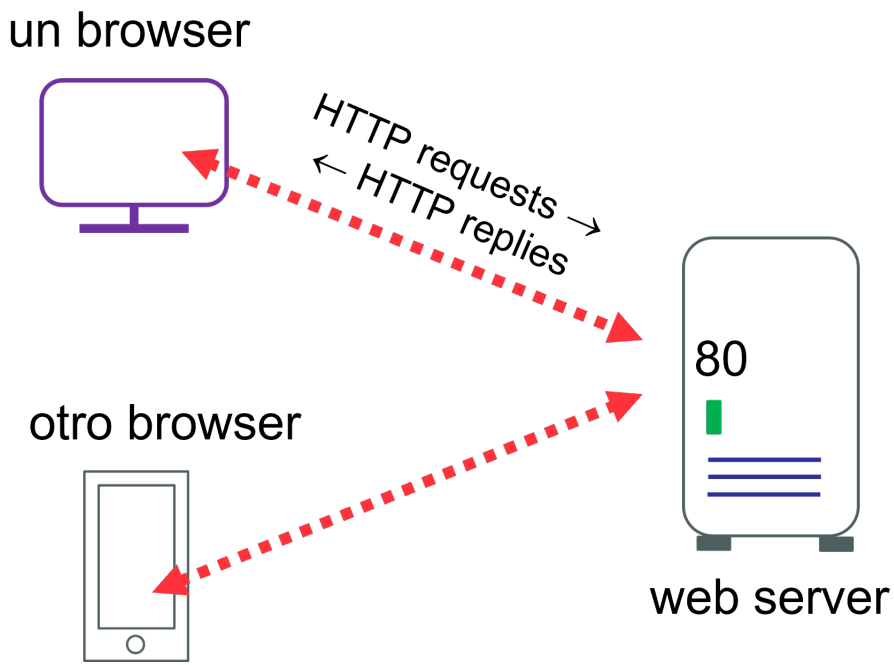


Hypertext Transfer Protocol

Definición

Protocolo de aplicación para solicitar y recibir documentos de hipertexto y otros recursos bajo demanda de los primeros.

- HTTP/1.0, 1996, RFC 1945
- HTTP/1.1, 2014, RFC 7235
- HTTP/2, 2015, RFC 9113
- HTTP/3, 2022, RFC 9114
- Cliente-Servidor
- Utiliza TCP, puerto 80
- Sin conservación de estado *(stateless)*



HTTP/1.0 - no persistente

cliente-web / browser



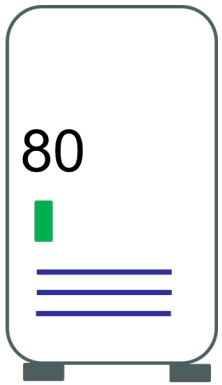
Cliente, puerto efímero, inicia la conexión hacia el servidor HTTP, Puerto 80

El servidor, que está esperando conexiones de clientes, acepta esta solicitud y le notifica.

El cliente envía la solictud del recurso por el socket.

El servidor envía el mensaje de respuesta conteniendo el objeto solicitado.

El servidor Cierra la conexión.



web server

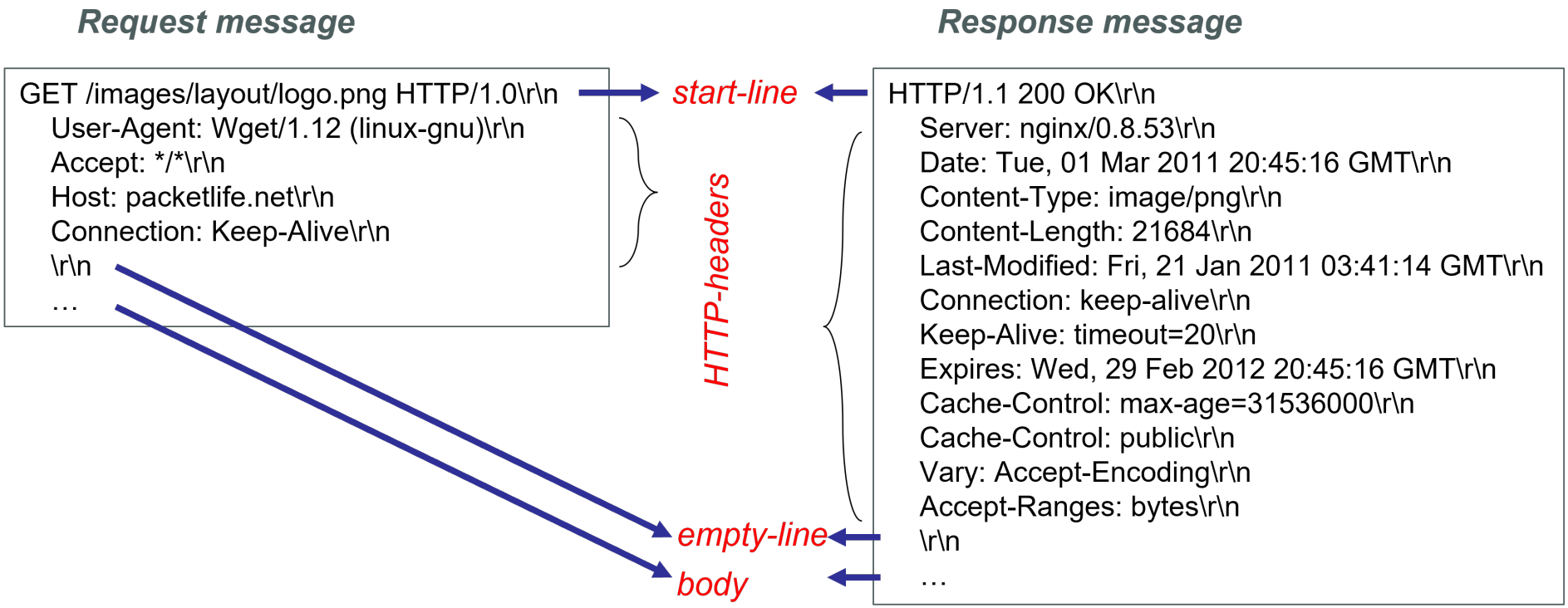
El cliente parsea el objeto respuesta. Si require otros Recursos, entonces inicia nuevas conexiones, repitiéndose los pasos anteriores.

HTTP/1.1 - persistente

- El servidor deja abierta la sesión para permitir nuevas solicitudes de objetos.
- El cliente puede solicitar más objetos a medida que son referenciados.
- Cualquier extremo puede cerrar la sesión.
- Minimiza el retardo comparado con HTTP 1.0
- Incorpora mecanismos para recibir datos faltantes si la sesión se cierra anticipadamente.

HTTP - mensajes

- Muchos tipos de mensajes request: GET, POST, HEAD...
- formato estándar, requests y responses: Start-line, HTTP-headers, empty-line y body.



Algunos Métodos Request

GET:

- Solicitud de recurso
- Puede incluir algo específico.

  www.sitios.com/lugares?canada

HEAD:

- Similar a GET, pero sólo requiere los encabezados.

POST:

- El cliente solicita se acepten los datos incluidos
- Son datos habitualmente no visibles al usuario

PUT:

- Solicitud de cargar de recurso en el servidor
- Crea o reemplaza el recurso

Respuestas

Tipos de respuestas a los Requests:

1xx Informational

- 100 Continue
- 101 Switching Protocols

2xx Successful

- 200 OK
- 202 Accepted

3xx Redirection

- 300 Multiple Choices
- 301 Moved Permanently

4xx Bad request

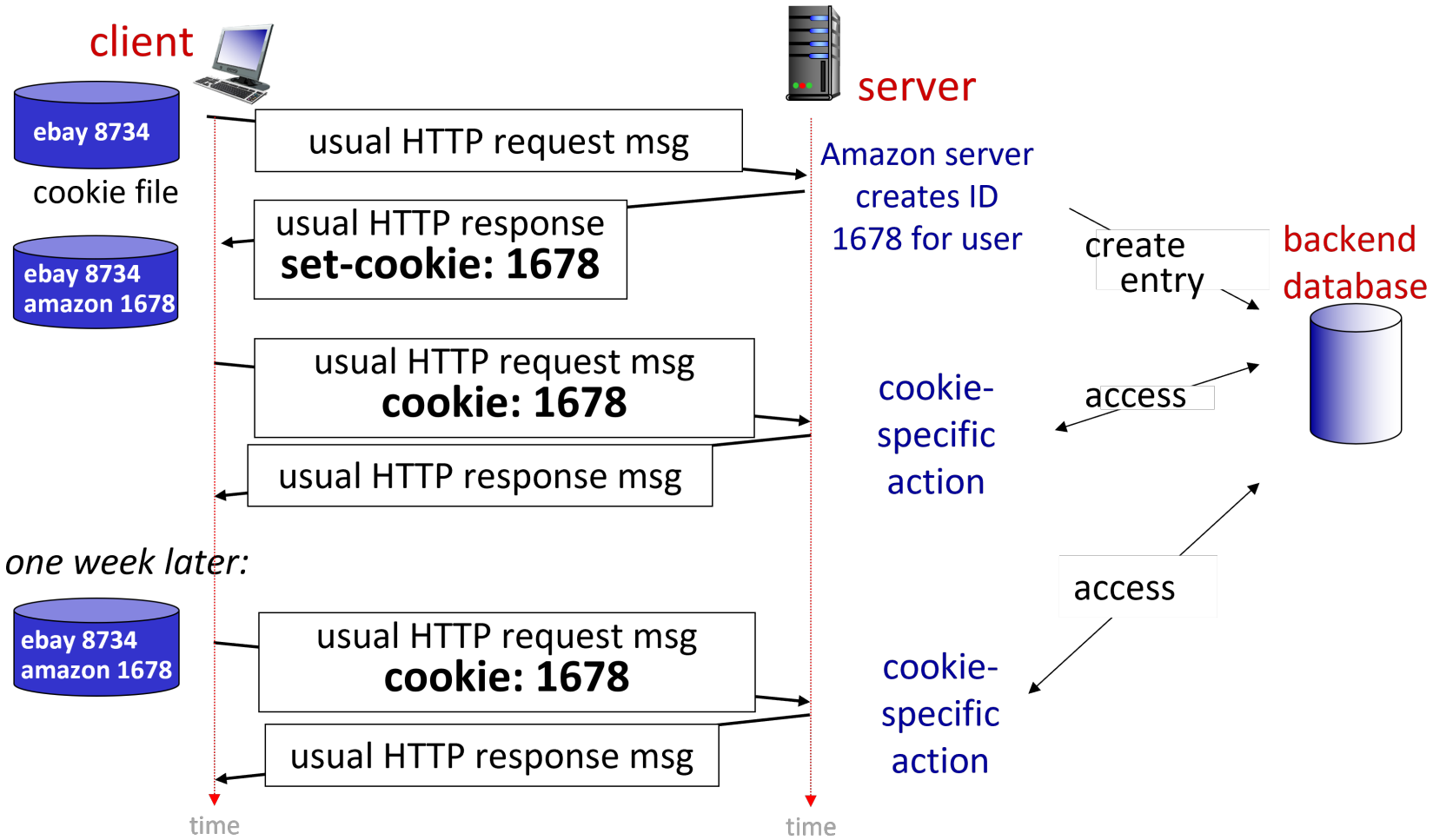
- 403 Forbidden
- 404 Not Found

5xx Server Error

- 501 Not Implemented
- 505 HTTP Version Not Supported

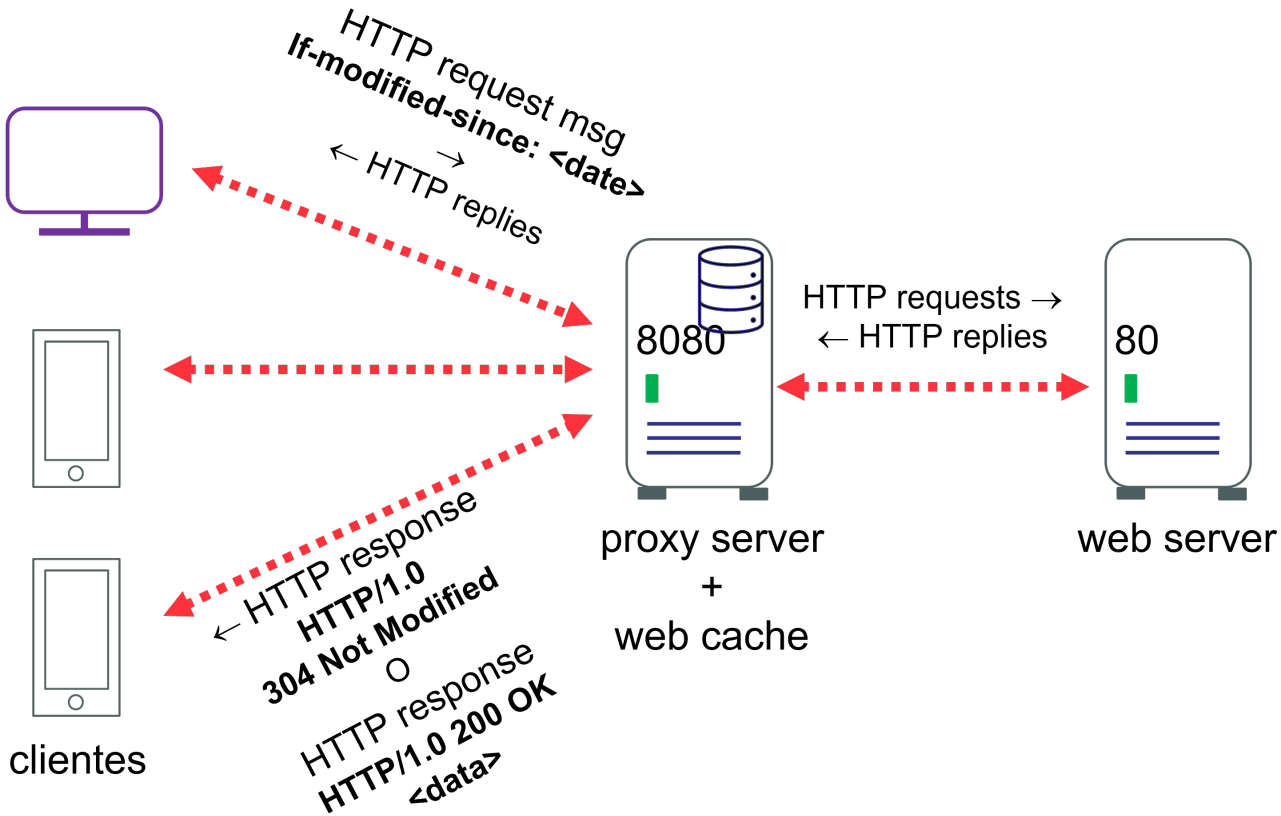
Conservar estado con cookies

Usos: autorizaciones, sesiones, carritos, acelerar entregas, etc.
Privacidad: conservación de información, cookies persitentes de terceros.



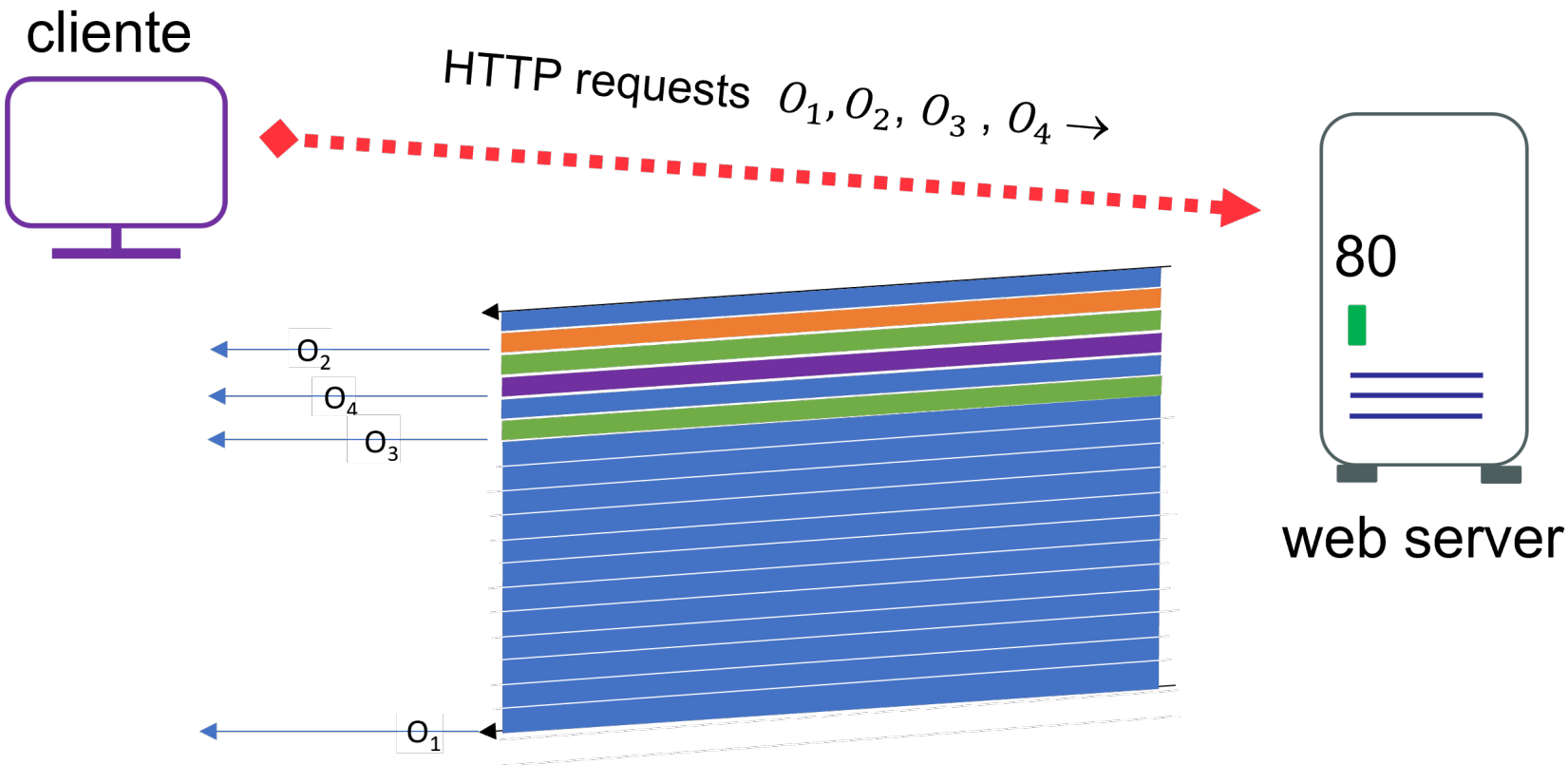
Proxy server + web cache

Objetivo: Atender los requests sin involucrar al servidor.
Ventajas: Mejorar el t. de respuesta, reducir el consumo de ancho de banda.
Corolario: Usando GET condicional se maximiza la operación.



HTTP/2 - entregas flexibles

- El cliente puede especificar la prioridad de los objetos solicitados.
- El servidor puede hacer push de objetos no solicitados.
- Los objetos se dividen en frames, los frames se entregan itercalados (interleaved) para mitigar el *HOL blocking*.



HTTP/3 - Quic

- Usa Quic como protocolo de transporte.
- Incorpora mecanismos de seguridad,
- y también de control de errores y congestión
- *Continuará...*

REST API

- REpresentational State Transfer Application Programming Interfaces
- Establece un conjunto de guías y límites que definen la arquitectura de desarrollo de una aplicación
  - Interface uniforme
  - Cliente-Servidor
  - Stateless
  - Cacheable
  - Sistema de capas
  - Código bajo demanda (opcional)

  Por ejemplo, las REST APIs basadas en HTTP usan los métodos HTTP estándares (GET, POST, PUT, DELETE, etc.) y las URIs (Uniform Resource Identifiers) para identificar recursos.

fin

++
31
--

RhymeZx 4y

I have an HTTP joke, if I POST it, you cannot GET it

joke/meme http

Favorite

